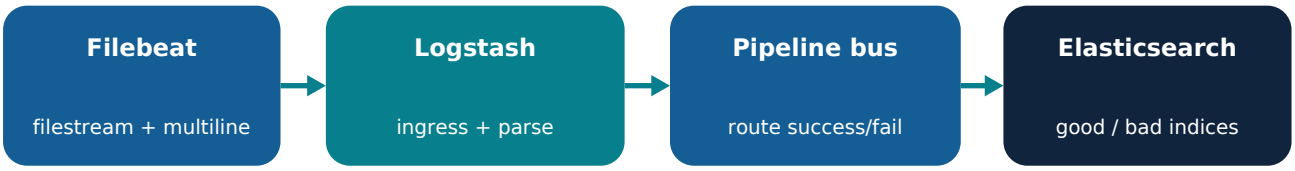
ELASTIC STACK RELIABLE INGESTION & DATA LIFECYCLE LABS

Filebeat | Logstash | Elasticsearch | Kibana



L3: PQ buffers output interruptions | DLQ handles eligible indexing rejects

LAB	DELIVERABLE	TIME
SETUP	Install and verify Filebeat and Logstash on Ubuntu	45-70 min
L3 | Reliable ingestion	Multiline, Grok/Date/Mutate, routing, queues, load test	150 min
L4 | Data management	Ingest simulation, mapping, templates, data streams, ILM rollover	110 min
Q2 | Query DSL	Match, term, range, bool filter with validated results	45 min
Extension	JSON/CSV/XML parsers, DLQ and licensed central management	45-60 min

Environment: Ubuntu lab virtual machine(s); self-managed Elastic Stack 8.19+ / 9.x; TLS-enabled Elasticsearch; Kibana Dev Tools; Filebeat and Logstash.

Prepared: 02 November 2026 | Audience: operations engineers, platform engineers and technical trainers

Includes a separate ZIP with complete configuration files, JSON request bodies, sample data and a bounded load generator.

1 | Lab scope, learning outcomes and boundaries

This manual continues from the uploaded Day 1 Ubuntu/KVM lab (L1, L2 and Q1). It adds explicit Logstash and Filebeat installation before L3. Elasticsearch and Kibana must already work on the Day 1 VMs. All index names, pipeline IDs and paths are prefixed for the lab. Run experiments only against a dedicated lab or disposable cluster; do not stop or reconfigure a shared production cluster.

EXERCISE	YOU WILL PROVE
L3.1	Select app log files; exclude debug files; consolidate stack traces as one event.
L3.2	Extract fields with Grok; normalize with Mutate and Date; send malformed events to quarantine.
L3.3	Connect four Logstash pipelines; distinguish PQ buffering, parser quarantine and DLQ.
L3.4	Create an isolated output interruption; inspect queue buildup; measure EPS and bottleneck.
L4.1	Simulate an ingest pipeline and verify each processor before indexing.
L4.2	Apply component/index templates, inspect explicit and newly inferred mapping fields.
L4.3	Index a data stream; roll it over; inspect ILM and backing indices.
Q2	Validate match, term, range and bool.filter with expected document IDs.

Prerequisite checklist

- Elasticsearch and Kibana versions are compatible; Logstash and Filebeat versions are on their supported matrix.
- You have sudo on the Logstash/Filebeat Ubuntu VM and a permitted test account with privileges for indexing, ingest pipelines, templates, data streams and ILM.
- Dedicated test Logstash pipeline ports 5044 (Beats input) and 9600 (local monitoring API) are not already in use.
- Allow at least 3 GB free disk for lab files and PQs; 4 GB RAM for a small Logstash training VM is a practical starting point.
- Security is enabled: use a valid Elasticsearch CA cert, HTTPS host, and credentials stored in the Logstash keystore.
- Snapshots are available before making lifecycle or security changes. The example ILM policy can delete lab data after one day.

TRAINER NOTE The companion Filebeat configuration shows example Logstash IP 192.168.122.21, which MUST be replaced with the actual VM address. Example cluster addresses are illustrations from Day 1, not actual machine IPs.

Trainer timing and pass criteria

TRACK	CHECKPOINT	PASS
L3	1 multiline stack trace + 1 malformed event	Good index + quarantine index
L3	Queue increases during interruption, drains after recovery	Metrics / screenshot and EPS report
L4	Six records, explicit + dynamic mapped fields	Template simulation + 2 backing indices
Q2	Expected result IDs for 4 queries	Saved Dev Tools responses

2A | Missing setup: install Logstash and Filebeat

Why this section is new: Day 1 set up three Elasticsearch 9.x nodes, Kibana, security and sample logs loaded directly with curl. It did not install Filebeat or Logstash. Complete the steps here BEFORE L3. No existing cluster data needs to be deleted.

A. Where exactly to install each component

DAY 1 VM / NEW VM	SERVICE	LAB ACTION
es-node1 (sample .11)	Elasticsearch + NEW Filebeat	Reads /var/tmp/elk-labs/logs/app-*.log; sends to Logstash
es-node2 (.12)	Elasticsearch	Keep as-is; do not install Logstash here
es-node3 (.13)	Elasticsearch	Keep as-is; do not install Logstash here
kibana (.20)	Kibana	Keep as-is; use browser and Dev Tools
logstash VM (sample .21)	NEW Logstash	Recommended 2 vCPU, 4 GB RAM, 20 GB disk, same private NAT network

TRAINER NOTE VM placement is a lab recommendation, not a requirement. If the physical host lacks memory for a fifth VM, Logstash may run on an existing well-provisioned VM such as the Kibana VM, but allocate adequate memory and change IP/host assumptions. Do not add Logstash to 3 GB Elasticsearch VMs without capacity review.

B. Verify Day 1 cluster before installing anything

Terminal on es-node1: Use your Day 1 HTTP CA and existing elastic account. Do not reinstall Elasticsearch or Kibana.

```
hostname -I
sudo systemctl is-active elasticsearch
curl --cacert /etc/elasticsearch/certs/http_ca.crt \
  -u elastic 'https://localhost:9200/_cat/nodes?v'
```

In Kibana Dev Tools (browser):

```
GET /
GET _cat/nodes?v
GET _cluster/health?pretty
```

Pass: exactly three joined Elasticsearch nodes, the cluster name insurance-lab, and an accessible Kibana console. Yellow health requires an explanation before ingesting training data; do not alter quorum/bootstrap settings.

C. Create or prepare the new Logstash Ubuntu VM

1. In Virtual Machine Manager, create a new Ubuntu Server guest (or reuse a dedicated available Ubuntu VM) with 2 vCPU, about 4 GB RAM and 20 GB disk. Attach its NIC to the same libvirt network as es-node1.

2. Set the guest hostname, discover its actual IP, and record it. 192.168.122.21 is an example only.

3. From Filebeat VM es-node1, confirm the VM is reachable. TCP 5044 will be tested after Logstash starts. Permit 5044 only from approved lab source IPs, not the Internet.

```
# Run on the NEW Logstash Ubuntu VM
sudo hostnamectl set-hostname logstash
hostname -I
sudo apt update
sudo apt install -y curl wget gnupg ca-certificates unzip netcat-openbsd openssh-server
sudo systemctl enable --now ssh
```

D. Install Logstash on the Logstash VM

Your Day 1 manual already established the Elastic 9.x APT repository on Elasticsearch/Kibana VMs. On a new Logstash VM, add the same 9.x repository with the same signing key. If the file already exists, inspect it rather than creating duplicate entries.

```
# LOGSTASH VM - INSTALL PACKAGE REPOSITORY
sudo apt update
sudo apt install -y curl wget gnupg ca-certificates
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch \
  | sudo gpg --dearmor \
  -o /usr/share/keyrings/elasticsearch-keyring.gpg
REPO='deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg]'
REPO="$REPO https://artifacts.elastic.co/packages/9.x/apt stable main"
echo "$REPO" | sudo tee /etc/apt/sources.list.d/elastic-9.x.list
sudo apt update
apt-cache policy logstash
sudo apt install -y logstash
```

Check the installed package and required paths. On a Debian installation: binary /usr/share/logstash/bin/logstash, settings /etc/logstash, pipeline configuration /etc/logstash/pipelines.yml, service logstash, monitoring API port 9600 when running.

```
dpkg -l logstash
/usr/share/logstash/bin/logstash --version
sudo test -f /etc/logstash/logstash.yml && echo 'settings found'
sudo test -f /etc/logstash/pipelines.yml && echo 'pipelines found'
sudo systemctl enable logstash
sudo systemctl status logstash --no-pager || true
```

TRAINER NOTE A package can be installed while the service is still inactive or has no useful pipeline configured. Do not mark installation failed just because 5044 is not listening before the pipeline files are installed. Start or restart Logstash in Section 4 after configuration and test.

2A continued | Install Filebeat on the existing es-node1

E. Install Filebeat only on the log-source VM

Use your existing Day 1 es-node1 VM. Because Day 1 created the Elastic 9.x repository there, do not repeat repository setup unless the repository file is missing. Do not replace the Elasticsearch configuration.

```
# ES-NODE1: Check existing repository
ls -l /etc/apt/sources.list.d/elastic-9.x.list
sudo apt update
apt-cache policy filebeat
sudo apt install -y filebeat
dpkg -l filebeat
filebeat version
sudo systemctl enable filebeat
sudo systemctl status filebeat --no-pager || true
```

Filebeat installation adds /etc/filebeat/filebeat.yml and a filebeat service. A default output.elasticsearch section may be enabled in the package configuration; the dedicated training file will replace it with output.logstash. Do not run filebeat setup dashboards through Logstash in this basic lab.

F. Transfer and extract companion ZIP to BOTH VMs

From your Ubuntu host, download and place the companion ZIP in your current directory. Copy it via SCP to both es-node1 and Logstash VM using your real username and IPs. Alternatively use VM console / shared media to copy the ZIP.

```
# ON UBUNTU HOST; replace USER and both addresses
scp Elastic_Stack_Day2_Revised_Lab_Files.zip USER@192.168.122.11:~/
scp Elastic_Stack_Day2_Revised_Lab_Files.zip USER@192.168.122.21:~/

# ON BOTH VMs, after receiving the ZIP
cd ~
unzip -o Elastic_Stack_Day2_Revised_Lab_Files.zip
cd ~/ELK_Lab_Files
pwd
ls filebeat logstash sample-data
```

TRAINER NOTE Do not type the example USER or sample IP literally. The actual IPs are from hostname -I on each guest. The files are the revised ZIP delivered with this PDF, not a file embedded in your Day 1 PDF.

G. Install basic Filebeat configuration on es-node1

```
# ES-NODE1 only; run from ~/ELK_Lab_Files
sudo install -d -m 0755 /var/tmp/elk-labs/logs
sudo cp sample-data/*.log /var/tmp/elk-labs/logs/
sudo cp /etc/filebeat/filebeat.yml /etc/filebeat/filebeat.yml.day1-backup
sudo cp filebeat/filebeat-lab.yml /etc/filebeat/filebeat.yml
sudo nano /etc/filebeat/filebeat.yml
# CHANGE output.logstash.hosts to actual logstash VM IP
sudo chown root:root /etc/filebeat/filebeat.yml
sudo chmod 0644 /etc/filebeat/filebeat.yml
sudo filebeat test config -c /etc/filebeat/filebeat.yml -e
```

The revised Filebeat sample explicitly uses prospector.scanner.fingerprint.length: 64 because the synthetic log file is under 1024 bytes. This reduced fingerprint size is useful for small teaching samples, but use a collision-resistant setting for production logs. Keep the input type filestream.

TRAINER NOTE The basic Filebeat-to-Logstash Beats input is unencrypted TCP 5044, suitable ONLY inside this isolated private VM training network using synthetic data. For real deployments enable Beats input SSL and set Filebeat output.logstash.ssl with a trusted CA and verified server identity. The Logstash-to-Elasticsearch leg remains TLS/HTTPS.

H. Confirm both packages and network reachability

```
# On Logstash VM
/usr/share/logstash/bin/logstash --version
sudo ss -lntp | grep ':5044' || echo 'Not listening yet'
# On Filebeat (es-node1) VM
filebeat version
ping -c 2 192.168.122.21
# AFTER Logstash is configured and running:
nc -vz 192.168.122.21 5044
sudo filebeat test output -c /etc/filebeat/filebeat.yml -e
```

If the Logstash VM has another IP, substitute it throughout. Connection refused before Logstash is configured is expected. A successful port test after starting the Beats input confirms TCP reachability, but the Filebeat output test confirms protocol communication.

2A continued | Secure connection and first end-to-end event

I. Copy the Elasticsearch HTTP CA to the Logstash VM

Logstash must trust the Day 1 Elasticsearch HTTPS endpoint. Only transfer the public CA certificate; do NOT transfer Elasticsearch private keys or remove HTTPS verification. These commands assume network access between nodes and a normal SSH login.

```
# ON ES-NODE1
sudo cp /etc/elasticsearch/certs/http_ca.crt ~/http_ca.crt
sudo chown "$USER":"$USER" ~/http_ca.crt
scp ~/http_ca.crt USER@192.168.122.21:~/http_ca.crt

# ON LOGSTASH VM
sudo install -d -m 0755 /etc/logstash/certs
sudo install -m 0644 ~/http_ca.crt /etc/logstash/certs/http_ca.crt
ls -l /etc/logstash/certs/http_ca.crt
```

Use an Elasticsearch HTTPS address whose hostname/IP is covered by its server certificate, usually your Day 1 es-node1 endpoint. Verify the hostname before the first test; do not work around a certificate mismatch with certificate verification disabled.

J. Create restricted training indexing credentials

In Kibana Stack Management / Security, create a role logstash_training_writer with index names training-l3-*, index privileges auto_configure, create_index, write, view_index_metadata and cluster privilege monitor if needed. Create native user logstash_training with only this role and a private password. This is separate from Day 1 lab_operator, which had only service-logs-lab permissions.

On the Logstash VM, populate the four Logstash keystore values. At each prompt provide the complete value. For ES_USER use logstash_training, ES_URL use https://your-es-node1:9200, ES_CA_CERT use /etc/logstash/certs/http_ca.crt, and ES_PASSWORD use the private password.

```
# ON LOGSTASH VM
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash create
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_URL
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_USER
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_PASSWORD
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_CA_CERT
sudo chown logstash:logstash /etc/logstash/logstash.keystore
sudo chmod 0640 /etc/logstash/logstash.keystore
```

TRAINER NOTE If the keystore already exists, skip the create command and use add. Do not record password or keystore contents in screenshots. For a strictly temporary private lab, an administrator account can be used for initial diagnostics only, then replaced with the scoped user.

K. Install the four pipelines and bring up services in order

On the Logstash VM, finish Sections 2B and 4: copy logstash files, configure pipelines.yml, enable the queue, validate the config, and start/restart Logstash. The main lab describes those commands in detail. On es-node1, only start Filebeat AFTER the Beats input listens on the chosen IP and port.

```
# LOGSTASH VM, after Section 4 configuration
sudo /usr/share/logstash/bin/logstash --path.settings /etc/logstash \
  --config.test_and_exit
sudo systemctl restart logstash
sudo systemctl status logstash --no-pager
sudo ss -lntp | grep ':5044'
curl -s http://127.0.0.1:9600/_node/pipelines?pretty

# ES-NODE1, when Logstash port 5044 is ready
sudo filebeat test config -c /etc/filebeat/filebeat.yml -e
sudo filebeat test output -c /etc/filebeat/filebeat.yml -e
sudo systemctl enable --now filebeat
sudo journalctl -u filebeat -n 40 --no-pager
```

L. Confirm that the flow reached Elasticsearch

Kibana Dev Tools:

```
GET training-l3-good/_count
GET training-l3-failed/_count
GET training-l3-good/_search?size=2
GET training-l3-failed/_search?size=2
```

On a clean first ingest, expected counts are 4 good events and 1 parser-quarantine event. You should find the stack trace preserved within one event. The precise counts may differ if data was ingested before or the sample was copied twice. In Kibana Discover, set a custom time range covering the sample timestamps rather than assuming Last 24 hours.

M. Troubleshooting installation failures

SYMPTOM	NEXT CHECK
Unable to locate package	Check elastic-9.x.list, apt update, GPG key and Internet/DNS
Service inactive	journalctl -u logstash / filebeat; validate YAML/pipeline first
No listener on 5044	Logstash pipeline config, startup exception, port collision
Filebeat output refuses connection	Replace example Logstash IP; check nc -vz and NAT/firewall
TLS certificate mismatch	Copy correct HTTP CA, use HTTPS URL matching server SAN
Indexing 403	Training Logstash user must write to training-l3-* indices
0 hits but services healthy	File selection, fingerprint, registry, ES counts, Discover time picker

2B | Environment discovery, staging and secure configuration

Step 1 - verify versions and health

From your Ubuntu terminal, run these where installed; use Kibana Dev Tools for Elasticsearch API requests. Record the returned major versions in the lab evidence.

```
# Run on each relevant VM, after Section 2A install
sudo systemctl status logstash --no-pager   # Logstash VM
filebeat version                            # es-node1 VM
sudo ss -lntp | grep -E ':(5044|5601|9200|9600)'
df -h /var/lib/logstash /var/tmp
```

```
GET /
GET _cluster/health
GET _nodes?filter_path=nodes.*.version
GET _license
```

Expected: reachable cluster, coherent node versions and enough disk. A single-node yellow cluster may simply have unassigned replicas; the L4 lab sets zero replicas only for its own indices.

Step 2 - create test paths and stage log data

```
# ES-NODE1; sample logs were staged in Section 2A
ls -lah /var/tmp/elk-labs/logs
# LOGSTASH VM; from ~/ELK_Lab_Files
sudo install -d -m 0750 /etc/logstash/lab
sudo cp logstash/{10,20,30,40}-*.conf /etc/logstash/lab/
sudo chown -R root:logstash /etc/logstash/lab
sudo chmod 0750 /etc/logstash/lab
sudo chmod 0640 /etc/logstash/lab/*.conf
```

The relative paths above refer to the unzipped companion pack. Substitute the actual directory where you extracted the ZIP. Copying to /etc/logstash requires sudo.

Step 3 - configure TLS and secret substitution

On the dedicated Logstash VM, the Elasticsearch HTTP CA has already been copied and the keystore populated in Section 2A. Do not repeat the create command if the keystore exists. Confirm that the trusted CA belongs to the Elasticsearch endpoint specified in ES_URL.

```
sudo install -d -m 0755 /etc/logstash/certs
# Example only when ES and Logstash share a VM:
sudo cp /etc/elasticsearch/certs/http_ca.crt \
  /etc/logstash/certs/http_ca.crt
sudo chmod 0644 /etc/logstash/certs/http_ca.crt
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash create
sudo /usr/share/logstash/bin/logstash-keystore \
  --path.settings /etc/logstash add ES_URL ES_USER ES_PASSWORD ES_CA_CERT
```

When prompted, enter values separately: ES_URL=https://<es-host>:9200, ES_USER=a lab account, ES_PASSWORD=its secret, ES_CA_CERT=/etc/logstash/certs/http_ca.crt. Restrict keystore file access so

the Logstash service account can read it. Prefer a scoped training user rather than a superuser.

> TRAINER NOTE If an existing keystore exists, do not overwrite it; add the four keys. The Logstash configuration uses ${ES_*} interpolation. Never commit secrets to the companion ZIP or screenshots.

3 | L3.1 - Filebeat selection and multiline boundaries

Objective: Select app-*.log files while excluding app-debug.log; correctly turn a Java exception and its stack trace into a single Filebeat event.

Step 1 - inspect sample log lines

File: sample-data/app-training.log

```
2026-11-02T09:00:00.123Z INFO [checkout] order_id=ORD-1001 user=asha duration_ms=23
  message="Checkout completed"
2026-11-02T09:00:01.456Z ERROR [checkout] order_id=ORD-1002 user=ravi duration_ms=412
  message="Payment gateway failed"
java.lang.IllegalStateException: gateway unavailable
    at com.example.Checkout.pay(Checkout.java:42)
    at com.example.Service.handle(Service.java:21)
2026-11-02T09:00:02.000Z WARN [catalog] order_id=ORD-1003 user=maya duration_ms=87
  message="Cache miss"
2026-11-02T09:00:03.000Z UNKNOWN_LINE_THAT_FAILS_GROK
2026-11-02T09:00:04.000Z INFO [checkout] order_id=ORD-1004 user=lee duration_ms=35
  message="Payment settled"
```

This data contains five application events: four valid messages (one with a 3-line stack trace) and one deliberately malformed header that still starts with a timestamp. A debug file and an unrelated misc.log file are present to verify file selection.

Step 2 - backup and set Filebeat configuration

```
sudo cp /etc/filebeat/filebeat.yml \
  /etc/filebeat/filebeat.yml.before-elk-labs
sudo cp filebeat/filebeat-lab.yml /etc/filebeat/filebeat.yml
sudo chown root:root /etc/filebeat/filebeat.yml
sudo chmod 0644 /etc/filebeat/filebeat.yml
```

File: filebeat/filebeat-lab.yml

```
filebeat.inputs:
  - type: filestream
    id: training_app_logs
    enabled: true
    paths:
      - /var/tmp/elk-labs/logs/app-*.log
    exclude_files: ['app-debug\.log$']
    prospector.scanner.check_interval: 2s
    prospector.scanner.fingerprint.length: 64
    parsers:
      - multiline:
          type: pattern
          pattern: '^[0-9]{4}-[0-9]{2}-[0-9]{2}T'
          negate: true
          match: after
          timeout: 2s
          max_lines: 100
    fields:
```

```
      training: l3
output.logstash:
  hosts: ['192.168.122.21:5044']
logging.level: info
```

Interpretation: a line beginning with an ISO-like timestamp starts a new message. Non-matching lines are appended to the prior event (negate: true, match: after). Filebeat performs multiline aggregation before shipping to Logstash; this is safer than combining mixed Beats streams inside Logstash.

Step 3 - validate and start Filebeat after Logstash is ready

```
sudo filebeat test config -c /etc/filebeat/filebeat.yml -e
sudo filebeat test output -c /etc/filebeat/filebeat.yml -e
sudo systemctl restart filebeat
sudo journalctl -u filebeat -n 35 --no-pager
```

If the test output fails because Logstash is not yet listening on 5044, finish Section 4 first, then repeat this step. Make a copy of the data before rerunning; Filebeat tracks offsets and may not resend unchanged lines.

Step 4 - validation questions

1. Run GET training-l3-good/_count and GET training-l3-failed/_count once Logstash is running; expect 4 good and 1 malformed event for a fresh first pass.

2. Search for order.id = ORD-1002 and inspect event.original; confirm the exception and both “at ...” lines are in the same event.

3. Confirm neither ORD-IGNORE nor ORD-IGNORE2 appears in the training indices. Record why glob paths and exclude_files behave differently.

4 | L3.2 - Build parsing and pipeline-to-pipeline routing

Objective: separate transport, transformation, good-event indexing and quarantine handling into independently observable Logstash pipelines.

Step 1 - architecture and routing

```
Filebeat --> lab_ingress [beats:5044]
                    | pipeline output: parse_bus
                    v
              lab_parse [Grok / Date / Mutate]
                    |                     |
                 success               parse failure
                    v                     v
             index_bus                quarantine_bus
                    |                     |
             lab_indexer           lab_quarantine
                [PQ]                    [PQ]
                    v                     v
          training-l3-good       training-l3-failed
```

Pipeline-to-pipeline buses are local to one Logstash process; they are not network endpoints. The downstream pipeline declares pipeline { address => ... }, while the sender uses pipeline { send_to => [...] }. Avoid cyclic dependencies.

Step 2 - dedicated pipelines.yml

File: logstash/pipelines.yml

```
# Copy these entries to a dedicated lab Logstash instance only.
- pipeline.id: lab_ingress
  path.config: /etc/logstash/lab/10-ingress.conf
  pipeline.workers: 1
- pipeline.id: lab_parse
  path.config: /etc/logstash/lab/20-parse.conf
  pipeline.workers: 2
- pipeline.id: lab_indexer
  path.config: /etc/logstash/lab/30-indexer.conf
  pipeline.workers: 2
  pipeline.batch.size: 125
  pipeline.batch.delay: 50
  queue.type: persisted
  queue.max_bytes: 64mb
  queue.page_capacity: 16mb
- pipeline.id: lab_quarantine
  path.config: /etc/logstash/lab/40-quarantine.conf
  pipeline.workers: 1
```

```
  queue.type: persisted
  queue.max_bytes: 64mb
  queue.page_capacity: 16mb
```

Copy this file to /etc/logstash/pipelines.yml only on a dedicated test instance, after backing up the original. The two output pipelines use persisted queues, each bounded to 64 MB. Queue sizing is per pipeline.

Step 3 - ingress and parser configuration

File: logstash/10-ingress.conf

```
input { beats { id => "lab_beats" port => 5044 } }
output { pipeline { send_to => ["parse_bus"] } }
```

File: logstash/20-parse.conf

```
input { pipeline { address => "parse_bus" } }
filter {
  grok {
    match => {
      "message" => '^%{TIMESTAMP_ISO8601:app_ts} %{LOGLEVEL:app_level} \[%{DATA:app_service}\]
        order_id=%{NOTSPACE:order_id} user=%{NOTSPACE:app_user} duration_ms=%{INT:duration_ms:int}
        message="%{DATA:app_message}"'
    }
    tag_on_failure => ["_lab_grok_failure"]
  }
  if "_lab_grok_failure" not in [tags] {
    date {
      match => ["app_ts", "ISO8601"]
      target => "@timestamp"
      tag_on_failure => ["_lab_date_failure"]
    }
    mutate {
      copy => { "message" => "[event][original]" }
```

```
      rename => {
        "app_level" => "[log][level]"
        "app_service" => "[service][name]"
        "app_user" => "[user][name]"
        "order_id" => "[order][id]"
        "duration_ms" => "[app][duration_ms]"
        "app_message" => "[app][summary]"
      }
      add_field => { "[labels][training_lab]" => "L3" }
    }
  }
  if "_lab_grok_failure" in [tags] or "_lab_date_failure" in [tags] {
    mutate {
      add_tag => ["training_parse_failed"]
      add_field => { "[failure][reason]" => "grok_or_date_failed" }
    }
  }
}
```

```
output {
  if "training_parse_failed" in [tags] {
    pipeline { send_to => ["quarantine_bus"] }
  } else {
    pipeline { send_to => ["index_bus"] }
  }
}
```

The parser extracts app_ts, level, service, order_id, user, duration and summary. Date assigns the application timestamp to @timestamp. Mutate creates structured fields and preserves the full multiline text in event.original. Any Grok/Date failure receives the training_parse_failed tag and is directed to quarantine; this is a deliberate application-level routing rule, not an automatic DLQ feature.

Step 4 - output pipelines

File: logstash/30-indexer.conf

```
input { pipeline { address => "index_bus" } }
output {
  elasticsearch {
    id => "lab_es_good"
    hosts => ["${ES_URL}"]
    user => "${ES_USER}"
    password => "${ES_PASSWORD}"
    ssl_certificate_authorities => ["${ES_CA_CERT}"]
    index => "training-l3-good"
    data_stream => false
    ilm_enabled => false
  }
}
```

File: logstash/40-quarantine.conf

```
input { pipeline { address => "quarantine_bus" } }
output {
  elasticsearch {
    id => "lab_es_failed"
    hosts => ["${ES_URL}"]
    user => "${ES_USER}"
    password => "${ES_PASSWORD}"
    ssl_certificate_authorities => ["${ES_CA_CERT}"]
    index => "training-l3-failed"
    data_stream => false
    ilm_enabled => false
  }
}
```

TRAINER NOTE The output plugin example uses ssl_certificate_authorities (recent Elastic versions). If an older installed plugin does not recognize it, check the installed plugin documentation and its equivalent CA parameter. Do not disable TLS verification.

Step 5 - enable Logstash monitoring and DLQ, then validate

File: logstash/logstash.yml additions

```
# Add only to the DEDICATED training Logstash /etc/logstash/logstash.yml.
api.http.host: "127.0.0.1"
api.http.port: 9600
dead_letter_queue.enable: true
# Persistent queues are configured per pipeline in pipelines.yml.
```

```
sudo cp /etc/logstash/pipelines.yml \
 /etc/logstash/pipelines.yml.before-elk-labs
sudo cp logstash/pipelines.yml /etc/logstash/pipelines.yml
# Add settings from logstash.yml-additions.txt into /etc/logstash/logstash.yml
sudo /usr/share/logstash/bin/logstash --path.settings /etc/logstash \
  --config.test_and_exit
sudo systemctl restart logstash
sudo journalctl -u logstash -n 60 --no-pager
curl -s http://127.0.0.1:9600/_node/pipelines?pretty
```

If the validation command warns that pipelines.yml is ignored, check whether Logstash is running with -f or centralized management enabled. Confirm four named pipelines in the monitoring response before starting Filebeat.

Step 6 - inspect parsed, failed and multiline records

```
GET training-l3-good/_count
GET training-l3-failed/_count
GET training-l3-good/_search
{
  "size": 10,
  "query": { "term": { "order.id.keyword": "ORD-1002" } },
  "_source": ["@timestamp", "log", "service", "app", "event.original"]
}
GET training-l3-failed/_search
{ "query": { "term": { "tags.keyword": "training_parse_failed" } } }
```

Because L3 uses automatically inferred mappings, some dynamic keyword paths vary by Elastic version. If a term query returns zero unexpectedly, inspect GET training-l3-good/_mapping or search using a matching available exact field. In most recent stacks, dynamically mapped strings have a .keyword subfield.

5 | L3.3 - Grok, JSON, CSV, XML and error handling

Step 1 - Grok and Date exercise

1. On a copy of 20-parse.conf, change the expected literal order_id= to order_code= in the Grok pattern only; rerun configuration validation and watch how some records move into training-l3-failed.

2. Restore the proper pattern. Compare app_ts (raw text) with @timestamp (date) in good records. Explain how using ingestion time instead of event time changes range searches.

3. Change duration_ms to a nonnumeric token in one newly appended record. Observe that Grok with %{INT:...:int} fails and the record is quarantined.

TRAINER NOTE Use unique new log lines for retests rather than overwriting an already-harvested file: Filebeat maintains registry offsets and identities. For a clean repeat, create a new app-run2.log file.

Step 2 - one-off format parsing micro-lab

This separate lab demonstrates JSON, CSV and XML Logstash filters without modifying the main four pipelines. It generates three records internally, parses each into a parsed structure and prints them to stdout. The generator count is one, so no production data is involved.

File: logstash/90-format-mini-lab.conf

```
# Run ONLY as an independent one-off pipeline, not in pipelines.yml.
input {
  generator {
    count => 1
    lines => [
      'JSON|{"order_id":"ORD-9","amount":75,"status":"paid"}',
      'CSV|ORD-10,settled,23.50',
      'XML|<order><id>ORD-11</id><status>pending</status></order>'
    ]
  }
}
filter {
  dissect { mapping => { "message" => "%{format}|%{payload}" } }
  if [format] == "JSON" {
    json { source => "payload" target => "parsed" }
  } else if [format] == "CSV" {
    csv {
      source => "payload"
```

```
      target => "parsed"
      columns => ["order_id", "status", "amount"]
      convert => { "amount" => "float" }
    }
  } else if [format] == "XML" {
    xml { source => "payload" target => "parsed" store_xml => true }
  }
}
output { stdout { codec => rubydebug } }
```

```
# Check that installed plugins include json/csv/xml/dissect:
sudo /usr/share/logstash/bin/logstash-plugin list | \
  grep -E '(logstash-filter-json|logstash-filter-csv|logstash-filter-xml|logstash-filter-dissect)'
# Run as a SEPARATE test process with a UNIQUE data directory:
sudo /usr/share/logstash/bin/logstash \
  --path.settings /etc/logstash \
  --path.data /var/tmp/elk-labs/logstash-formats \
  -f /etc/logstash/lab/90-format-mini-lab.conf
```

Before running, copy the one-off file to /etc/logstash/lab/. In the stdout event, confirm that JSON becomes parsed.order_id / amount, CSV amount is numeric, and XML creates a parsed object. Stop with Ctrl+C after the output appears; this one-off example does not terminate automatically.

Step 3 - parsing failure vs DLQ vs PQ

CASE	DESTINATION	WHY
Malformed input syntax	training-l3-failed	Explicit tag-and-route rule in lab_parse.
Output unavailable / retryable failure	Persistent queue (if accepted)	Queues unacknowledged events pending output recovery; capacity bounded.
Eligible Elasticsearch bulk item rejection	Logstash dead-letter queue	Typically per-document 400/404 failures or conditional evaluation errors; DLQ must be enabled.

A Logstash DLQ is not a general catch-all for Grok failures or HTTP connection outages. Transport-level Elasticsearch output failures normally cause retries; they do not automatically become DLQ entries. A DLQ entry must be explicitly inspected, corrected and replayed using the dead_letter_queue input.

Optional - DLQ inspection and replay template

File: logstash/95-dlq-replay-example.conf

```
# OPTIONAL, run in its own process after an actual Elasticsearch bulk item 400/404.
# Replace path with your dedicated Logstash instance's path.data/dead_letter_queue.
input {
  dead_letter_queue {
    path => "/var/lib/logstash/dead_letter_queue"
    pipeline_id => "lab_indexer"
    commit_offsets => true
  }
}
filter {
  mutate { add_tag => ["dlq_replayed"] }
  # Add a case-specific field fix BEFORE reindexing. Do not replay unchanged.
}
output { stdout { codec => rubydebug { metadata => true } } }
```

An actual DLQ exercise requires deliberately producing an Elasticsearch non-retryable bulk item rejection (for example, a mapping conflict in a separate test index). Do not claim a DLQ pass merely because the malformed Grok test succeeded. Verify a file under path.data/dead_letter_queue/lab_indexer/, inspect the reason and apply a field correction before reindexing. The supplied replay template prints records only, so it is non-destructive.

6 | L3.4 - Backpressure, persistent queues and load

Step 1 - capture baseline pipeline metrics

From the dedicated Logstash VM, capture snapshots before sending extra load. The HTTP metrics API listens on localhost:9600; no Elasticsearch login is required for this local endpoint when configured as in the lab.

```
curl -s http://127.0.0.1:9600/_node/stats/pipelines?pretty \
  > /var/tmp/elk-labs/baseline.json
curl -s http://127.0.0.1:9600/_node/stats/pipelines/lab_indexer?pretty
curl -s http://127.0.0.1:9600/_node/stats/pipelines/lab_ingress?pretty
```

Record events.in, events.out, events.duration_in_millis and queue statistics where exposed. Newer versions can expose flow rates such as input_throughput, output_throughput and queue_backpressure; field presence varies by version.

Step 2 - generate bounded, fresh workload

```
python3 scripts/make_load.py --count 2000 --pad-bytes 0 \
  --output /var/tmp/elk-labs/logs/app-load.log
# Wait for Filebeat's scan + multiline timeout, then record metrics:
curl -s http://127.0.0.1:9600/_node/stats/pipelines?pretty \
  > /var/tmp/elk-labs/after-load.json
```

Compute average throughput for a measured interval: EPS = (events.out_after - events.out_before) / elapsed_seconds. Repeat the test with 2,000-5,000 fresh events and identical test conditions after recovery. Do not assume wall-clock command execution time equals ingestion completion time.

Step 3 - controlled output interruption

1. On a dedicated test Elasticsearch endpoint used exclusively by this lab, temporarily stop that endpoint using sudo systemctl stop elasticsearch. Do not stop a shared cluster node or any cluster with production responsibilities.

2. On the Logstash VM, append a bounded load with python3 scripts/make_load.py --count 10000 --pad-bytes 4096 --output /var/tmp/elk-labs/logs/app-load.log. This can create roughly tens of MB of synthetic data; monitor free space.

3. Repeatedly capture /_node/stats/pipelines/lab_indexer?pretty. Observe persistent queue occupancy / events_count, output retries and, if capacity is reached, upstream backpressure. A blocked output may not fill a large queue with too few events.

4. Restart the dedicated test endpoint with sudo systemctl start elasticsearch. Wait for recovery; confirm queue occupancy decreases and successful output counts rise. Avoid terminating Logstash abruptly during the drill.

5. Use GET training-l3-good/_count plus pipeline event counters as a consistency check; account for initial baseline and any retries. Report whether the bottleneck was Filebeat, Logstash filters/CPU or Elasticsearch output.

TRAINER NOTE If you cannot safely stop Elasticsearch, skip the interruption and use a specially isolated test endpoint/approved network-block drill instead. Never run OS-wide firewall rules or stop production services just to demonstrate backpressure.

Step 4 - controlled tuning (one variable at a time)

RUN	WORKERS	BATCH.SIZE	BATCH.DELAY	EVIDENCE
A baseline	2	125	50 ms	EPS / CPU / queue
B	1	125	50 ms	Repeat same load
C	2	250	50 ms	Repeat same load
D	2	125	10 ms	Repeat same load

Change only lab_indexer settings in pipelines.yml, restart the dedicated Logstash instance and rerun with newly named files. Output throughput may be limited by Elasticsearch latency and disk I/O; increasing workers or batch size does not guarantee improvement. Record latency, CPU saturation, dropped/failed events and output backpressure alongside EPS.

7 | L3 extension - centrally managed pipelines (licensed)

Centralized Logstash Pipeline Management is a subscription feature. It is not available in every license tier. Verify your installed Elastic subscription in Stack Management > License Management before presenting the feature.

1. Show the Kibana management interface for Logstash pipelines, when exposed by the current license and user role.

2. Create a small demonstration pipeline under a unique ID in a non-production instance or demonstrate the editor in read-only mode.

3. On a separate dedicated Logstash instance, configure xpack.management.enabled: true, the Elasticsearch hosts and registered pipeline IDs, along with approved authentication. Restart that instance.

4. Edit the pipeline centrally, validate its managed deployment and capture configuration history/screenshots if the UI supports it.

5. Explain the consequence: once centralized management is enabled on that Logstash instance, local pipelines.yml and path.config are inactive. Do not enable it on the local-pipeline L3 instance.

```
# Example settings concept; NOT for the local L3 instance:
xpack.management.enabled: true
xpack.management.elasticsearch.hosts: ["https://es-lab:9200"]
xpack.management.pipeline.id: ["centrally_managed_demo"]
# Add compatible secure authentication and TLS settings.
# Requires feature entitlement and suitable privileges.
```

Decision exercise - Logstash vs Elasticsearch ingest nodes

CRITERION	LOGSTASH	INGEST PIPELINE
Sources	Beats, Kafka, files, databases and plugins	Documents already reaching Elasticsearch
Processing	Rich conditionals, Grok, Date, CSV, XML, external outputs	Lightweight processors, Grok, Set, Convert, Script
Buffering	Persistent queues and backpressure management	No built-in equivalent to Logstash disk PQ
Routing	Multiple pipelines / outputs	Index routing and processor logic
Operational cost	Extra nodes / JVM / plugin lifecycle	Adds work to ingest-capable Elasticsearch nodes

Selection rule: use Elasticsearch ingest pipelines for simple transforms close to indexing; use Logstash when heterogeneous input handling, routing, PQ/DLQ capabilities or more complex plugin transformations justify the added tier. Both can be used together.

8 | L4.1 - Ingest pipeline simulation

Objective: build a deterministic ingest pipeline, simulate it without writing documents, and verify fields before attaching it to a data stream.

Step 1 - create processor pipeline

```
PUT _ingest/pipeline/training-enrich
{
  "description": "Training - simple Elasticsearch ingest enrichment",
  "processors": [
    {
      "set": {
        "field": "labels.training_lab",
        "value": "L4"
      }
    },
    {
      "uppercase": {
        "field": "log.level",
        "ignore_missing": true
      }
    },
    {
      "set": {
```

```
        "field": "ingest_status",
        "value": "processed"
      }
    }
  ]
}
```

The set processor adds labels.training_lab; uppercase normalizes log.level; set adds ingest_status. For records missing log.level, uppercase skips that field instead of failing.

Step 2 - simulate without indexing

```
POST _ingest/pipeline/training-enrich/_simulate?verbose=true
{
  "docs": [
    {
      "_index": "training-events-demo",
      "_id": "s1",
      "_source": {
        "@timestamp": "2026-11-02T10:00:00Z",
        "message": "Payment timeout",
        "log": {
          "level": "error"
        },
        "service": {
          "name": "checkout"
        },
        "duration_ms": 520
      }
    },
```

```
    {
      "_index": "training-events-demo",
      "_id": "s2",
      "_source": {
        "@timestamp": "2026-11-02T10:01:00Z",
        "message": "Login success",
        "log": {
          "level": "info"
        },
        "service": {
          "name": "identity"
        },
        "duration_ms": 45
      }
    }
  ]
}
```

Inspect the docs[].doc._source result and per-processor status. Both lowercase log.level values should become uppercase, and each record should contain labels.training_lab = L4 and ingest_status = processed. This simulation alone does not create any index or store documents.

Step 3 - failure investigation

1. Temporarily change uppercase.field to a nonexistent nested field and remove ignore_missing. Simulate again; inspect the processor error.

2. Restore the pipeline before continuing. Save the successful simulation response as L4 evidence.

3. Describe how on_failure handlers could tag a bad input without silently dropping it.

9 | L4.2 - Static/dynamic mappings and composable templates

Templates must exist before a matching data stream or index is created. The explicit mappings define known field types; the dynamic mapping policy allows Elasticsearch to infer new fields for previously unseen keys.

Step 1 - create the tiny ILM policy

```
PUT _ilm/policy/training-events-ilm
{
  "policy": {
    "phases": {
      "hot": {
        "actions": {
          "rollover": {
            "max_docs": 3
          }
        }
      },
      "delete": {
        "min_age": "1d",
        "actions": {
          "delete": {}
        }
      }
    }
```

```
  }
}
```

The hot phase uses max_docs = 3 for quick demonstration; the delete action begins at min_age = 1d. The manual never waits a full day for deletion. This policy must never be attached to non-lab data.

Step 2 - create mapping component template

```
PUT _component_template/training-mappings-v1
{
  "template": {
    "mappings": {
      "dynamic": true,
      "properties": {
        "@timestamp": {
          "type": "date"
        },
        "message": {
          "type": "text"
        },
        "log": {
          "properties": {
            "level": {
              "type": "keyword"
            }
          }
```

```
        },
        "service": {
          "properties": {
            "name": {
              "type": "keyword"
            }
          }
        },
        "duration_ms": {
          "type": "integer"
        },
        "ingest_status": {
          "type": "keyword"
        }
      }
    }
  }
}
```

Fields explicitly defined here are static types. dynamic: true allows new fields such as environment to be added when actual data is indexed. For strict schemas, switch to dynamic: strict in a separate experimental index to observe rejection on unknown fields.

Step 3 - create settings component template

```
PUT _component_template/training-settings-v1
{
  "template": {
    "settings": {
      "index.number_of_shards": 1,
      "index.number_of_replicas": 0,
      "index.lifecycle.name": "training-events-ilm",
      "index.default_pipeline": "training-enrich"
    }
  }
}
```

This maps the ILM policy and default ingest pipeline onto each new backing index. A single-node lab sets number_of_replicas = 0; make an intentional resiliency choice in a multi-node real cluster.

Step 4 - compose the index template and simulate resolution

```
PUT _index_template/training-events-template
{
  "index_patterns": [
    "training-events-*"
  ],
  "priority": 300,
  "data_stream": {},
  "composed_of": [
    "training-mappings-v1",
    "training-settings-v1"
  ],
  "_meta": {
    "owner": "training",
    "purpose": "L4 data stream lab"
  }
}
```

```
POST _index_template/_simulate/training-events-template
GET _index_template/training-events-template
GET _component_template/training-mappings-v1
```

Confirm combined mappings, lifecycle settings and the data_stream declaration. The training-events-* pattern is deliberately disjoint from the built-in logs-*-* family; priority 300 resolves competing matches within this isolated namespace.

10 | L4.3 - Data streams, ILM and rollover

Step 1 - create the data stream

```
PUT _data_stream/training-events-demo
GET _data_stream/training-events-demo
GET _cat/indices/.ds-training-events-demo*?v&expand_wildcards=all
```

Expected: one hidden backing index with a generation suffix similar to 000001. The exact date in that backing-index name depends on creation time. Data streams require documents with an @timestamp field and create-only indexing semantics.

Step 2 - index deterministic sample records

In Kibana Console, run the following requests. A full six-record bulk payload is also included as elasticsearch/07-sample-documents.ndjson in the ZIP for curl-based ingestion. Use unique document IDs to avoid 409 conflicts on repeat.

```
PUT training-events-demo/_create/l4-1
{
  "@timestamp": "2026-11-02T10:00:00Z",
  "message": "payment timeout connecting to gateway",
  "log": { "level": "error" },
  "service": { "name": "checkout" },
  "duration_ms": 520,
  "environment": "training"
}
PUT training-events-demo/_create/l4-2
{
  "@timestamp": "2026-11-02T10:01:00Z",
  "message": "payment completed",
  "log": { "level": "info" },
  "service": { "name": "checkout" },
  "duration_ms": 90
}
PUT training-events-demo/_create/l4-3
```

```
{
  "@timestamp": "2026-11-02T10:02:00Z",
  "message": "cart timeout waiting on stock",
  "log": { "level": "warn" },
  "service": { "name": "checkout" },
  "duration_ms": 310
}
```

Step 3 - force manual rollover for a controlled demonstration

```
POST training-events-demo/_rollover
GET _data_stream/training-events-demo
GET _cat/indices/.ds-training-events-demo*?v&expand_wildcards=all
GET .ds-training-events-demo*/_ilm/explain?expand_wildcards=all
```

Expected: two backing indices, with the newer generation as the current write index. ILM may roll over on its own after a poll once max_docs has been reached; if that happened already, inspect generations before forcing a further rollover. A forced manual rollover is a validation shortcut, not proof that the periodic ILM scheduler executed the policy.

Step 4 - index the remaining three documents into the new write index

```
PUT training-events-demo/_create/l4-4
{"@timestamp":"2026-11-02T10:03:00Z",
 "message":"database connection timeout","log":{"level":"error"},
 "service":{"name":"billing"},"duration_ms":720}
PUT training-events-demo/_create/l4-5
{"@timestamp":"2026-11-02T10:04:00Z",
 "message":"login successful","log":{"level":"info"},
 "service":{"name":"identity"},"duration_ms":45}
PUT training-events-demo/_create/l4-6
{"@timestamp":"2026-11-02T10:05:00Z",
 "message":"payment rejected by issuer","log":{"level":"error"},
 "service":{"name":"checkout"},"duration_ms":430}
```

```
POST training-events-demo/_refresh
GET training-events-demo/_count
GET training-events-demo/_mapping
GET _data_stream/training-events-demo
GET .ds-training-events-demo*/_ilm/explain?expand_wildcards=all
```

Expected: exactly six documents if the lab starts from a clean stream. The environment field, absent from the explicit component mapping, should have a dynamic mapping in the backing index that first received l4-1. The ingest pipeline should have normalized every log.level to uppercase.

Step 5 - compare hot / warm / cold / frozen / delete

TIER / PHASE	ROLE	LAB IMPLEMENTATION
Hot	Active indexing and frequent queries	Used: rollover policy; write index remains hot.
Warm	Less frequent updates/reads, less costly storage	Concept only; no dedicated warm nodes.
Cold	Infrequently queried historical data	Concept only; tier allocation requires suitable nodes.
Frozen	Very infrequent search, often searchable snapshots	Concept only; architecture and licensing vary.
Delete	Remove expired backing indices per policy	Policy defined (1d), not waited for.

TRAINER NOTE A one-node lab does not simulate physical tier movement just because an ILM phase is named warm or cold. Check data-node roles, allocation requirements, snapshot repositories and licensed features before demonstrating real tier transitions.

11 | Q2 - Query DSL comparison and validation

Objective: identify ERROR records in a time/duration window while avoiding unnecessary scoring for exact filters. Run these in Kibana Dev Tools after all six L4 records are indexed.

Q2a - analyzed match query

```
GET training-events-demo/_search
{
  "size": 20,
  "query": { "match": { "message": "payment" } },
  "_source": ["message", "log.level", "service.name"]
}
```

Expected IDs: l4-1, l4-2 and l4-6. Why: match analyzes text in the message text field. Scores may differ based on relevance.

Q2b - term query on an exact keyword

```
GET training-events-demo/_search
{
  "size": 20,
  "query": { "term": { "log.level": "ERROR" } },
  "_source": ["message", "log.level"]
}
```

Expected IDs: l4-1, l4-4 and l4-6. Why: log.level is explicitly mapped as keyword and the ingest processor uppercase-normalized the stored value.

Q2c - bool.filter with keyword, numeric range

```
GET training-events-demo/_search
{
  "size": 20,
  "query": {
    "bool": {
      "filter": [
        { "term": { "log.level": "ERROR" } },
        { "term": { "service.name": "checkout" } },
        { "range": { "duration_ms": { "gte": 400, "lte": 600 } } }
      ]
    }
  },
  "_source": ["message", "duration_ms"]
}
```

Expected IDs: l4-1 and l4-6. Why: exact conditions run in filter context; they do not contribute to relevance score, which is unnecessary for operational filtering.

Q2d - selective scoring plus filter

```
GET training-events-demo/_search
{
  "size": 20,
  "query": { "bool": {
    "must": [ { "match": { "message": "payment" } } ],
    "filter": [ { "term": { "service.name": "checkout" } } ]
  } }
}
```

Expected IDs: l4-1, l4-2 and l4-6. The must clause scores message relevance; the service filter narrows results without affecting that score. Compare _score with Q2c.

Q2 written questions

1. Why is term on a text field usually the wrong choice for exact full-string matching?
2. Why would term log.level: error return no results when the indexed value is ERROR?
3. What is the operational benefit of moving environment, level and time restrictions into bool.filter?
4. Modify Q2c to use an @timestamp range from 10:00:00Z to 10:06:00Z and preserve expected IDs.

12 | Measurements, evidence and scoring rubric

Evidence checklist - screenshots or saved JSON results

- [E1] Elastic Stack version and cluster health snapshot; lab host / node list.
- [E2] Filebeat input configuration showing glob, exclude_files and multiline rules.
- [E3] Example event.original containing the complete Java stack trace as one event.
- [E4] Grok, Date, Mutate and four pipeline .conf files, plus pipelines.yml.
- [E5] One quarantine record with training_parse_failed and its original malformed header.
- [E6] Before/during/after Logstash pipeline and PQ metrics for the interruption.
- [E7] EPS calculation, workers/batch tuning comparison and a written bottleneck diagnosis.
- [E8] Ingest simulation result showing fields before/after processors.
- [E9] Component templates, index template, ILM policy and template simulation result.
- [E10] Data stream metadata, two backing indices and ILM explain snapshot.
- [E11] Dynamic field mapping for environment alongside explicit keyword/text/numeric mapping.
- [E12] Four query bodies, response IDs, hit totals and a brief explanation of scoring.

Acceptance rubric (100 points)

AREA	POINTS	CRITERIA
L3 selection / multiline	15	Correct file inclusion/exclusion and 1 combined stack trace.
L3 parsing / routing	20	4 parsed records, 1 quarantined record, valid configs.
L3 resilience / performance	20	Measured queue behavior + EPS and bottleneck analysis.
L4 ingest / mappings	15	Successful simulation, component/index templates, dynamic field.
L4 data stream / ILM	15	6 documents, 2 generations and lifecycle status evidence.
Q2 Query DSL	15	Four validated queries with correct expected record IDs.

Passing recommendation: at least 75/100, with no security violation or unapproved disruption. For central management and DLQ extensions, award separate bonus marks; both may depend on licensing or access to a deliberately controlled failure environment.

Measurement worksheet

RUN	IN BEFORE	OUT AFTER	ELAPSED SEC	EPS	QUEUE / BOTTLENECK
Baseline					
1 worker					
Batch 250					
Output blocked					
Recovered					

13 | Troubleshooting and safe reset

SYMPTOM	INVESTIGATION / ACTION
No events in Elasticsearch	Confirm port 5044, Filebeat test output, Logstash four pipelines, TLS CA and credentials.
Too few events	Check Filebeat registry offsets; create a new app-* file for replays, and wait for multiline timeout.
Stack trace split	Check filestream parsers and timestamp-start regex; avoid Logstash multiline codec for multiple Beats streams.
_grokparsefailure	Compare exact log format with anchored Grok pattern; inspect message and multiline boundaries.
Config syntax error	Use Logstash --config.test_and_exit; check quoted braces, file ownership and plugin installation.
Indexing 403	Assign test roles for indices, pipelines, templates, ILM and stream management; do not disable security.
Template ignored	Template must precede data stream creation; inspect index_patterns and priority.
Rollover not observed	Check nonempty backing index, ILM poll interval, policy status and manual rollover response.
DLQ directory empty	A Grok parse failure and an unreachable Elasticsearch node do not, by themselves, generate DLQ entries.
Queue never fills	The load may be below configured capacity; inspect events_count and queue occupancy, not assumptions.
Logstash settings ignored	Check -f launch parameter and xpack.management.enabled; central mgmt overrides local pipeline configs.

Cleanup - only when evidence has been exported

```
# Kibana Dev Tools -- deleting a data stream is destructive:
DELETE _data_stream/training-events-demo
DELETE _index_template/training-events-template
DELETE _component_template/training-mappings-v1
DELETE _component_template/training-settings-v1
DELETE _ingest/pipeline/training-enrich
DELETE _ilm/policy/training-events-ilm
DELETE training-l3-good
DELETE training-l3-failed
```

Restore the backed-up Filebeat and Logstash configuration, restart only the dedicated lab services, and remove /var/tmp/elk-labs when no longer needed. Do not delete /var/lib/logstash queues before stopping Logstash and confirming all needed data was delivered. A DLQ requires separate safe retention and disposition.

Trainer sign-off

Lab participant: _____ Date: _____ Score: _____ / 100

Observed bottleneck: _____ Recovery demonstrated: [ ] Yes [ ] No

14 | Official reference material

Installation links: https://www.elastic.co/docs/reference/logstash/installing-logstash and https://www.elastic.co/docs/reference/beats/filebeat/setup-repositories . Filebeat filestream fingerprint guidance: https://www.elastic.co/docs/reference/beats/filebeat/filebeat-input-filestream .

Filebeat filestream multiline: https://www.elastic.co/docs/reference/beats/filebeat/multiline-examples

Logstash pipeline-to-pipeline: https://www.elastic.co/docs/reference/logstash/pipeline-to-pipeline

Persistent queues: https://www.elastic.co/docs/reference/logstash/persistent-queues

Dead-letter queues: https://www.elastic.co/docs/reference/logstash/dead-letter-queues

Logstash performance: https://www.elastic.co/docs/reference/logstash/tuning-logstash

Logstash centralized management: https://www.elastic.co/docs/reference/logstash/logstash-centralized-pipeline-management

Composable templates: https://www.elastic.co/docs/manage-data/data-store/templates

ILM create policy: https://www.elastic.co/docs/manage-data/lifecycle/index-lifecycle-management/configure-lifecycle-policy

Ingest pipeline simulate API: https://www.elastic.co/docs/api/doc/elasticsearch/operation/operation-ingest-simulate

Query/filter context: https://www.elastic.co/docs/reference/query-languages/query-dsl/query-filter-context

Exact term query: https://www.elastic.co/docs/reference/query-languages/query-dsl/query-dsl-term-query

References checked 02 November 2026. Endpoint and plugin behavior may vary by deployed Elastic Stack version. Validate features against the installed version and subscription before training.